Faculty of Informatics Engineering
Department of Software Engineering
& Information System

كلية هندسة المعلوماتية
قسم هندسة البرمجيات
و نظم المعلومات

AutoTest & DocGen Manager

Intelligent System for Automated Testing and Documentation

Graduation Project 2 — Part 1: Introduction and Theoretical Study

Prepared by

Areej Nooraldeen Kamar Aldiab Ola Najibeh Wiaam Alouni

Supervised by

Dr. Riad Sonbol Eng. Anas Abdulaziz

2026-2027

Table of Contents

Introduction..... 3
I.1 Project Context..... 3
I.2 Problem Statement..... 3
I.3 Significance..... 5
I.4 Project Objectives..... 5
I.5 Scope and Constraints..... 6
I.6 State of the Art..... 8
I.7 Proposed Solution..... 15
Chapter 1: Theoretical Study..... 21
1.1 Introduction..... 21
1.2 Software Testing Fundamentals..... 21
1.3 Automated Test Generation..... 21
1.4 Test Adequacy: Coverage and Mutation Testing..... 22
1.5 Safe Execution of Generated Code..... 22
1.6 Regression Test Selection and Flaky Tests..... 22
1.7 Abstract Syntax Trees and Static Analysis..... 23
1.8 API Evolution and Breaking Changes..... 23
1.9 Code–Documentation Inconsistency..... 24
1.10 Logical Code Explanation and Summarization..... 25
1.11 Large Language Models in Software Engineering..... 25
1.12 Evaluation Methodology..... 26
1.13 Summary..... 27
References..... 28

Introduction

This part of the report presents the scientific problem addressed in the second phase of the graduation project, its objectives, the state of the art, and the proposed solution. Chapter 1 then presents the theoretical background on which the work is built.

I.1 Project Context

AutoTest & DocGen Manager is a web platform that analyzes source code and uses Large Language Models (LLMs) to automate documentation and quality-assurance tasks. The first phase of the project (Graduation Project 1) delivered the code-analysis pipeline (Python AST and Tree-sitter processors, a unified class diagram, and a dependency graph), a multi-agent documentation generator (HighLevelAgent, LowLevelAgent and VerifierAgent), and a first version of a conflict-detection module with three modes: Code vs Code, Code vs Documentation, and Full Analysis.

The second phase extends the platform along two directions:

- **Automated testing.** A new testing module that generates pytest unit, edge-case and integration tests with an LLM from the AST and class diagram, validates and executes them in an isolated sandbox, and reports code coverage over time (requirements RE-FR-40 to RE-FR-54).
- **Accurate and verifiable analysis.** Improving the precision of the Code vs Code and Code vs Documentation analyses, and proving the accuracy of both the conflict detection and the generated logical code explanations (High-Level and Low-Level) with a reproducible evaluation on real, labeled data instead of LLM self-assessment.

I.2 Problem Statement

The phase-one system can describe code, but it cannot yet test it, and neither its ability to detect inconsistencies nor the factual accuracy of its generated explanations has been measured in a scientifically valid way. Four concrete problems motivate this phase.

I.2.1 The Testing Gap

Although the platform's name promises automated testing, phase one focused on documentation. Writing tests manually is costly, so tests are often missing for the code paths that matter most. LLMs can generate tests, but their raw output is unreliable: in Meta's industrial deployment of TestGen-LLM, only 75% of generated test cases built correctly and 57% passed reliably [3]. Generated tests may fail to compile, assert hallucinated behavior, or be flaky. In addition, executing generated code that depends on the user's project is a security and stability risk if it is not isolated. A useful testing feature must therefore not only generate tests, but also validate, repair, execute safely, and measure them.

I.2.2 Limitations of Code vs Code Conflict Detection

Inspection of the current implementation and a controlled baseline run revealed the following limitations:

- **Signature-only comparison.** The unified class diagram stores only method names and a textual signature. Default values, variadic parameters (*args, **kwargs) and keyword-

only parameters are not extracted, so a change such as `timeout=30 → timeout=5` is not detected at all.

- **No behavioral evidence.** The LLM-based semantic analyzer receives class diagrams without method bodies, so it cannot detect a behavioral change hidden behind an unchanged signature.
- **Inaccurate severity.** Every signature difference is treated as breaking; adding an optional parameter, which the AexPy catalogue classifies as compatible, is reported as critical.
- **Robustness defect.** A schema-validation error caused every structural change to raise an exception, so the structural path failed whenever a real difference existed.

I.2.3 Limitations of Code vs Documentation Detection

The current Code vs Documentation checker relies on string matching between element names and the documentation text. It does not understand semantics: in the baseline run, a docstring stating that a method raises KeyError while the code returns None was not detected. At the same time, every undocumented method is reported as an inconsistency, which mixes documentation coverage with documentation correctness and inflates false alarms.

I.2.4 The Evaluation Gap

In phase one, conflict detection was evaluated by an LLM-as-a-Judge, which assigned 85/100. Such a score is a model judging a model; it provides neither a ground truth nor precision and recall. The literature shows why this matters: C4RLLaMA, a fine-tuned state-of-the-art detector, achieved a precision of 0.83 on a synthetic dataset but only 0.08 on real-world code-documentation pairs [6]. Claims about accuracy must therefore be supported by real, manually labeled data and standard metrics.

The same gap affects the generated logical explanations. In phase one they were scored by a four-layer framework in which 40% of the score came from an LLM judge, and the factual-accuracy layer only checked whether class and method names from the AST appear in the text; the overall result was 0.739 (GOOD). A name can appear in a sentence that describes its behavior wrongly, so this score does not show whether the statements in an explanation are true. The literature confirms the risk: about one fifth of GPT-4-generated method comments contained demonstrably inaccurate statements, and similarity metrics and existing consistency detectors showed no significant relationship with this factual accuracy [10].

I.2.5 Research Questions

The problem is formulated as follows: *how can a code-analysis platform generate trustworthy tests and detect breaking or behavioral changes and code-documentation inconsistencies with high precision and a low false-alarm rate, while proving the accuracy of these analyses and of its generated explanations on real, labeled data?* This leads to five research questions:

1. **RQ1:** How reliable (compilable, passing, non-flaky) and how effective (coverage) are LLM-generated tests when combined with syntax validation, a bounded repair loop, and sandboxed execution?

2. **RQ2:** Does combining deterministic AST analysis with targeted LLM analysis detect breaking and behavioral changes more accurately than the signature-only baseline, while reducing the number of LLM calls?
3. **RQ3:** Does local, per-element LLM classification with external filtering (LCEF) improve the precision and flag rate of Code vs Documentation detection compared with string matching?
4. **RQ4:** How factually accurate are the generated High-Level and Low-Level explanations at the statement level, and does an AST-based verification score agree with human judgment?
5. **RQ5:** How does the improved system compare with published LLM baselines on real, labeled datasets?

I.3 Significance

Practical significance. Inconsistencies are common in real software: DocPrism confirmed incorrectness inconsistencies in at least 11% of 1,991 documented functions from 20 popular projects [6]. Breaking changes are also frequent, including in AI-generated code: agentic pull requests introduced potential breaking changes in 6.72% of refactoring patches and 9.35% of chore patches [8]. A tool that flags these issues with few false alarms, and that also produces executable tests, directly reduces maintenance risk.

Academic significance. Plain LLM prompting flags more than 90% of functions as inconsistent [6], which makes tools unusable. This project studies how deterministic program analysis, evidence checks, and filtering can make LLM-based analysis precise, and it contributes a reproducible evaluation protocol (frozen baseline, pinned model, two-annotator labeling with Cohen's κ).

I.4 Project Objectives

I.4.1 Primary Objectives

- **PO1 – Automated test generation and execution.** Generate unit, edge-case and integration tests for every analyzable file of a single file, ZIP project, or GitHub repository; validate and repair them (up to 3 attempts); execute them in an isolated sandbox; and report coverage per file and per project (RE-FR-40 to RE-FR-46). *Measured by* the generation-failure rate, the acceptance rate, and the line/branch coverage and mutation score of accepted tests (environment errors are reported separately), compared with Pynguin run in the same sandbox on the same projects, with the TestGen-LLM figures (75% built, 57% passed reliably) as a reference point.
- **PO2 – Hybrid Code vs Code detection.** Detect structural changes deterministically using AexPy change patterns and severity levels, compare method bodies in every case, and call the LLM only for behavioral candidates. *Measured by* agreement with AexPy ground truth (click 7.1.2→8.0.4, urllib3 1.26.15→2.0.7) and the share of changes resolved without an LLM call.
- **PO3 – LCEF-based Code vs Documentation detection.** Classify each element against its own documentation snippet into Consistent, Direct_Mismatch, Over_Promise or Under_Promise, filter under-promises externally, and apply a confidence threshold.

Measured by precision, flag rate and F1 on DocPrism data, and recall, specificity and Pair-wise Fix Precision (PFP) on the Cascade dataset.

- **PO4 – Verified explanation accuracy.** Prove the accuracy of the generated logical explanations by labeling their individual claims (behavior, parameters, return values, exceptions, control flow) as correct or incorrect against the code, and by adding a deterministic AST-based check to the VerifierAgent so that explanations are verified with evidence rather than by an LLM judge. *Measured by* statement-level factual precision, the share of explanations containing at least one inaccurate statement (the measure used by Kang et al. [10]), entity-level hallucination F1 (as in ETF [23]), and the agreement between the AST-based score and human labels.

I.4.2 Secondary Objectives

- **SO1 – Reproducible evaluation.** Freeze the phase-one system as a baseline, use a deterministic LLM client (pinned model, temperature 0, no cache), and build a ground truth labeled independently by two annotators with Cohen's $\kappa \geq 0.60$.
- **SO2 – Competitive accuracy.** Exceed the balanced-split precision of the plain LLM baselines reported by Cascade (0.53–0.59), while keeping recall above Cascade's 0.21 [4].
- **SO3 – Test maintenance features.** Track coverage per commit (RE-FR-47), select affected tests through the dependency graph (RE-FR-48), detect flaky tests at the first successful run (RE-FR-49), and enforce per-user AI quotas with no automatic retry when exceeded (RE-FR-50).

I.5 Scope and Constraints

In the current phase (MVP), the complete testing pipeline (generation → validation → isolated execution → coverage) targets **Python with pytest** exclusively; other languages will be added later through a strategy layer. For the detection and explanation tracks, Python is the primary language, and Java is supported for evaluation on the Cascade dataset through the existing Tree-sitter-based processor. Table 1 lists the approved testing requirements, which continue the numbering of the phase-one requirements (RE-FR-01 to RE-FR-39), together with their actors.

ID	Requirement (summary)	Actor	Priority
RE-FR-40	Generate unit tests with AI for every analyzable file of the input (single file, ZIP project or GitHub repository), based on the AST and class-diagram data	Developer	Very High
RE-FR-41	Generate edge-case tests covering boundary values (null, empty, zero, negative) and the exceptions identified in the code	Developer	Very High
RE-FR-42	Generate integration tests for groups of related classes or files identified through the dependency graph and class-diagram relations	Developer	Very High
RE-FR-43	Skip any file that cannot be analyzed (syntax error, unsupported language, binary or empty file) without stopping the rest, and log the file name and reason	Developer	Medium
RE-FR-44	Validate the syntax of every generated test file before saving and request an automatic correction on failure, up to 3 attempts	Developer	Very High
RE-FR-45	Install declared dependencies in a controlled preparation step, then run the tests in an isolated sandbox with network disabled and limits	Developer	Very High

ID	Requirement (summary)	Actor	Priority
	on time, memory and CPU		
RE-FR-46	Execute the generated tests and report code coverage per file and per project, with details of every failed test (name, error type, message)	Developer / Reviewer	Very High
RE-FR-47	Track and visualize coverage over time: by default for the latest commit of each push, and for individual commits on demand	Developer / Reviewer	Medium
RE-FR-48	Select and run only the tests affected by a code change using the dependency graph	GitHub Webhook	Medium
RE-FR-49	Detect flaky tests whose results vary across repeated runs without code changes, and exclude them from official coverage	Developer	Medium
RE-FR-50	Enforce a per-user AI quota (requests, and input and output tokens counted separately) with a clear error when exceeded	Developer / Admin	Medium
RE-FR-51	Evaluate generated tests with mutation testing and report a mutation score	Developer	Low
RE-FR-52	Update a generated test automatically when its function signature changes in a minor, non-breaking way	Developer	Low
RE-FR-53	Scan project code for common security-risk patterns (hardcoded secrets, unsafe queries) and report them by severity	Developer / Reviewer	Low
RE-FR-54	Test the web frontend with a Playwright browser agent that interacts with the running application, captures screenshots and analyzes them with AI	Developer	Low

Table 1: Functional requirements of the testing module

The approved scope also fixes the operational constraints of the testing pipeline for the current phase (Table 2). These constraints keep the cost under control and separate environment problems from real test failures.

#	Constraint	Rule in the current phase
1	Execution environment and target language	The full pipeline runs first on Python and pytest only; other languages are added later through a strategy layer.
2	Model input and context	Generation relies on the AST and class diagram only; sending the whole project code to the model is not allowed, to save resources and keep accuracy high.
3	Integration and isolation limits	Integration between project modules is in-process only; external dependencies are isolated with test doubles or skipped with a limited depth.
4	Network and security	Network access is allowed temporarily during dependency installation only; tests always run without network and with strict time, memory and CPU limits inside the sandbox.
5	Environment errors	A failure to install dependencies or prepare the environment is classified as an Environment Error and is not counted as a failure of the test code.
6	Unanalyzable files	A file with a syntax error or binary content is skipped automatically and the reason is logged, without stopping the rest of the project.
7	Generation validation	A test is presented as ready only after it passes inside the sandbox on the current code. A syntax-validation failure after the 3 automatic repair attempts, or a test that does not pass on the current code, is classified as a Generation Failure and is not presented as a ready test.
8	Coverage calculation	Coverage is computed by default for the latest commit of each push; older

#	Constraint	Rule in the current phase
		commits are computed only on explicit request.
9	Affected-test execution	Only the elements affected according to the dependency graph are run, with a limited search depth to avoid waste.
10	Flaky-test check	The flakiness check runs only at the first successful run, with a fixed and limited number of re-runs to avoid wasting time.
11	AI quota management	A strict per-user quota (requests and input/output tokens) is applied; when it is exceeded, a clear error is returned with no automatic retry.

Table 2: MVP operational scope and constraints of the testing module

Constraints. The system depends on LLM APIs accessed through OpenRouter, which introduces latency, rate limits and model availability risks; evaluation runs therefore pin a specific model and record its version. Generated tests can only be executed for projects whose dependencies can be installed in the preparation step; installation failures are reported as environment errors. Code vs Documentation analysis is post hoc (static) and does not execute the analyzed code.

I.6 State of the Art

I.6.1 Similar Systems for Automated Testing

The team examined eight systems that represent the range of approaches to automated test generation, coverage tracking and test intelligence, and evaluated each against the generation, verification and coverage loop of the testing module. They fall into four groups: AI test generators (CoverUp, Qodo, Diffblue, GitHub Copilot), coverage tracking (Codecov), test intelligence for CI (CloudBees Smart Tests), and contrast systems that test outside the source structure (Keploy and mabl). These systems are reviewed to position the project; none of them is used inside it. Statements about commercial systems are based on vendor documentation, project repositories and public articles reviewed in October 2026.

- **CoverUp** [2] is an open-source research tool that generates high-coverage pytest regression tests for Python. It prompts an LLM, runs the generated tests, measures coverage and re-prompts with the lines and branches that remain uncovered; it accepts only tests that pass in isolation and raise coverage, detects flaky tests and can run inside Docker. It reaches a per-module median line+branch coverage of 80% versus 47% for CodaMosa, and 89% overall versus 77% for MuTAP. It is the closest technical baseline, but it is a command-line tool whose single objective is coverage.
- **Qodo** [26] (formerly CodiumAI) offers Qodo Gen, an IDE assistant that generates tests from an analysis of code behavior, and Qodo Cover, an open-source coverage-driven agent that commits a test only if running it raises coverage. It is IDE-oriented and does not document coverage history, isolation, AI budgets or failure classification.
- **Diffblue** [27] offers Diffblue Cover, a commercial Java unit-test generator based on symbolic AI and bytecode analysis rather than LLMs, and the newer Testing Agent for Java and Python (pytest), which orchestrates third-party coding agents, verifies that each test compiles, passes and adds coverage, and keeps only passing tests. It is the closest commercial counterpart, but it depends on a licensed agent platform and a local CLI or server.

- **Codecov** [28] is a hosted service that tracks coverage uploaded from CI, with history over time, coverage per commit, pull-request deltas and thresholds. It neither generates nor executes tests, and it is the reference model for the commit-based coverage history of RE-FR-47.
- **CloudBees Smart Tests** [29] (based on Launchable) predicts which tests are most likely to catch a failure for a change using diffs, history and metadata, groups related failures and deprioritizes flaky tests. It does not generate or run tests, and its selection is statistical, whereas the project's affected-test selection (RE-FR-48) is deterministic and derived from the dependency graph.
- **GitHub Copilot** [30] writes unit tests interactively inside the editor. It provides no automated validation pipeline, coverage measurement, result storage or project-level service, and serves as the general AI baseline.
- **Keploy** [31] records real API calls and database queries and replays them as tests with generated mocks, and **mabl** [32] is a cloud platform for UI-level test automation with auto-healing locators. Both derive tests from runtime traffic or UI interaction rather than source structure, and are included as contrasts and as references for future UI testing (RE-FR-54).

Table 3 compares the systems with the testing module, whose column follows the approved requirements RE-FR-40 to RE-FR-54. By the team's decision, a generated test is presented as ready only after it passes inside the sandbox on the current code; a test that does not pass is classified as a generation failure.

Feature / capability	CoverUp	Qodo	Diffblue	Codecov	CloudBees	Copilot	Keploy	mabl	Ours
Unit test generation (RE-FR-40)	✓	✓	✓	✗	✗	◐	◐	✗	✓*
Edge-case test generation (RE-FR-41)	◐	✓	?	✗	✗	◐	✗	✗	✓*
Integration test generation (RE-FR-42)	?	?	?	✗	✗	◐	✓	◐	✓*
Skips unparsable files, logs reason (RE-FR-43)	?	?	?	?	?	?	?	?	✓*
Structure-aware context (AST / bytecode / RAG)	◐	✓	✓	✗	◐	◐	✗	✗	✓*
Syntax validation & self-repair (RE-FR-44)	✓	◐	◐	✗	✗	✗	✗	◐	✓*
Isolated sandbox execution (RE-FR-45)	◐	?	◐	✗	✗	?	◐	◐	✓*
Validation on current code before acceptance	✓	✓	✓	✗	✗	✗	◐	✗	✓*
Code coverage measurement (RE-FR-46)	✓	✓	✓	✓	✗	✗	✓	✗	✓*
Failure classification	✗	?	?	✗	◐	✗	✗	◐	✓*
Coverage history over commits (RE-FR-47)	✗	?	?	✓	✗	✗	◐	?	✓*
Affected-test selection (RE-FR-48)	?	?	?	✗	✓	✗	?	?	✓*
Flaky test detection (RE-FR-49)	✓	?	?	✗	✓	✗	?	◐	✓*
AI usage quota / budget control (RE-FR-50)	?	?	◐	✗	?	?	?	?	✓*
Mutation testing (RE-FR-51)	?	?	✓	✗	✗	✗	?	?	◐*
Test maintenance on code change (RE-FR-52)	?	◐	✓	✗	✗	✗	?	✓	◐*
Project-level service / multi-source input	✗	◐	◐	✓	✓	✗	◐	✓	✓*
CI/CD integration	?	✓	✓	✓	✓	◐	✓	✓	✓

Feature / capability	CoverUp	Qodo	Diffblue	Codecov	CloudBees	Copilot	Keploy	mabl	Ours
Documentation generation	✗	◐	✗	✗	✗	✗	✗	✗	✓*

*Legend: ✓ full support · ◐ partial support · ✗ not available · ? not documented in the reviewed sources · * planned in the approved requirements*

Table 3: Feature comparison of similar systems for automated testing

System	Main strengths	Critical gaps	Relevance (team estimate)
CoverUp	Coverage-guided LLM loop; executes and verifies every test; flaky detection; Docker option; strong published evaluation	CLI only; coverage is the sole objective; no coverage history, affected-test selection, asynchronous API or documentation	High – closest technical baseline for the generate–execute–cover loop
Qodo	Behavior-based generation; coverage acceptance gate; RAG context; many languages	IDE-focused; no documented run-result or coverage-history management; no UML or outdated-doc detection	High – validates generation tied to coverage
Diffblue	Verified regression tests (compile, pass, add coverage); baseline coverage; planning; Java and Python	Licensed; relies on third-party agent platforms; no documented coverage history, flaky handling or docs; Cover is Java/bytecode only	High – closest commercial counterpart
Codecov	Coverage history per commit, PR deltas, thresholds, public API	Does not generate or run tests; depends on uploaded reports	Medium-High – reference for RE-FR-47
CloudBees Smart Tests	Predictive selection, failure grouping, flaky detection, test-health reports	No generation, execution or coverage; statistical selection	Medium-High – reference for RE-FR-48 and RE-FR-49
GitHub Copilot	Convenient inline test scaffolding in popular IDEs	Manual; no validation, coverage, storage or tracking	Medium – general AI baseline
Keploy	Traffic-based API tests, mocks, offline replay, unified coverage	Needs a running app and traffic; not source-structure aware; no docs	Low-Medium – contrast
mabl	Auto-healing UI tests, cross-browser cloud execution	UI level only; closed source; no source-level generation	Low – contrast and future UI work

Table 4: Limitations analysis of similar systems (relevance is the team's judgment, not a measured result)

Scientific foundation. Beyond the systems above, the team reviewed the research on LLM-based unit test generation and ranked it by its practical value for the testing module: how directly it informs the requirements, how much it contributes to proving that LLM-generated tests can be trusted, and whether it can be reproduced in experiments. Table 5 summarizes the seven core works; their reported results come from different languages and datasets and are therefore not directly comparable with each other.

#	Work	Technique and input	Language / data	Reported result	What the project takes
1	CoverUp (FSE 2025) [2]	LLM prompted with code and uncovered lines; generated tests are run and accepted only if they pass and raise coverage; flaky handling	Python, pytest	Median line+branch coverage 80% vs 47% for CodaMosa	No acceptance without execution; coverage measurement; flaky handling; isolation (RE-FR-45, 46, 49)

#	Work	Technique and input	Language / data	Reported result	What the project takes
2	TestPilot (TSE 2024) [1]	Signature, implementation and documentation examples; failing tests re-prompted with the error message	JavaScript, 25 npm packages	Median statement 70.2% / branch 52.8% coverage	Repair prompt = failing test + error message (RE-FR-44)
3	TestGen-LLM (FSE 2024 Industry) [3]	Chain of acceptance filters: build, pass reliably, add coverage	Kotlin, industrial (not public)	75% built, 57% passed reliably, 25% added coverage; 73% accepted by engineers	Filtering as evidence of trust (RE-FR-44, 46); reference figures only
4	ChatUniTest (FSE 2024 Demo) [33]	Adaptive focal context around the unit under test; generation–validation–repair	Java	Highest overall line coverage among the compared tools (vs TestSpark, EvoSuite)	Structured focal context and validation–repair design (RE-FR-40, 44)
5	Pynguin (SSBSE 2020) [13]	Search-based (evolutionary) generation without an LLM; exports pytest tests	Python modules	Evolutionary algorithms outperform random generation; dynamic typing remains hard	Non-LLM Python baseline in experiments only
6	MuTAP (IST 2024) [34]	LLM tests improved with surviving mutants fed back into the prompt	Python (HumanEval, Refactory)	Mutation score 93.57% on synthetic buggy code; up to 28% more faulty programs detected	Mutation score as a quality measure (RE-FR-51, evaluation)
7	CodaMosa (ICSE 2023) [14]	Pynguin search; LLM queried when coverage stalls	Python modules	Improves on Pynguin; lower than CoverUp	Related work and comparison point for CoverUp

Table 5: Research on LLM-based unit test generation, ranked by practical value for the project

Evaluation datasets. Three benchmarks are used for evaluation rather than for implementation. ClassEval provides 100 Python classes (412 methods) with about 33 tests per class; hiding its tests and generating new ones from the code allows pass, coverage and mutation results to be compared at class level [22]. ULT contains 3,909 real-world Python functions designed to reduce data contamination; LLM-generated tests reached only 41.32% accuracy on ULT compared with 91.79% on the older TestEval benchmark, which shows how older benchmarks can inflate results [35]. MultiFileTest (formerly ProjectTest) contains 20 multi-file projects for each of Python, Java and JavaScript and studies manual and self error-fixing; it is the closest benchmark to the project-level input of the platform and to the repair loop of RE-FR-44 [36]. Randoop and EvoSuite are mentioned only as historical context.

Design decisions derived from the review.

- **Input context.** Following the focal-context idea of ChatUniTest, the prompt contains the code of the unit under test together with AST and class-diagram context for its dependencies. The whole project source is never sent to the model, which is consistent with constraint 2 and keeps the cost bounded while still allowing edge cases and exceptions to be derived from the code (RE-FR-41).

- **What is taken from CoverUp.** The project adopts execution-based acceptance, coverage measurement and flaky handling. CoverUp's coverage-guided regeneration of uncovered lines is not part of the current requirements, whose repair loop is limited to syntax (RE-FR-44), and is recorded as future work.
- **Measuring trust correctly.** Because a test is presented as ready only after it passes on the current code, the pass rate of accepted tests is 100% by construction and proves nothing on its own. The evaluation therefore reports the generation-failure rate, the acceptance rate, the coverage of accepted tests and their mutation score, since a passing test may still assert incorrect current behavior (the oracle problem). Pynguin is run in the same sandbox on the same projects as a non-LLM baseline, and the TestGen-LLM figures serve only as a reference for trust.

Gap for the testing module. In the reviewed sources, generators (CoverUp, Qodo, Diffblue) do not document coverage tracking across commits, while trackers and selectors (Codecov, CloudBees Smart Tests) do not generate tests, and no single system combines them. None of the generators documents a classification that separates environment errors, generation failures and test failures, or a per-user AI quota with separate request and token limits. Affected-test selection in the reviewed tools is statistical, whereas the project uses the dependency graph with a bounded depth. Finally, test generation is not combined with documentation generation in the same platform. The project therefore proposes a single web service that generates unit, edge-case and integration tests from AST and class-diagram context, runs them in a sandbox, presents a test as ready only after it passes on the current code, stores commit-based coverage history, executes affected tests through the dependency graph, excludes flaky tests, classifies failures, enforces AI quotas, and integrates all of this with documentation generation.

I.6.2 API Breaking-Change Detection

AexPy detects API breaking changes in Python packages by building a detailed API model through hybrid analysis (dynamic reflection and static AST analysis), comparing API pairs with 42 change patterns, and grading each change with four levels: High, Medium, Low and Compatible. It matches parameters by kind (positional, keyword, variadic), resolves aliases, and downgrades changes to members whose names start with an underscore. On a benchmark of 61 known breaking changes it detected 53 (recall of about 87%), more than PyCompat and pidiff; on the latest versions of 45 packages it reported 433 high- or medium-level changes, 405 of which were true, and developers confirmed 31 of the 63 previously undocumented changes it reported [7]. Because it needs to import the package, it requires an installable project.

Safer Builders reconstructs syntactically valid before/after code for each diff hunk, parses both with Python's AST, and applies 17 patterns derived from AexPy. Manual validation found 93.6–95.7% true positives with Cohen's $\kappa = 0.79$. The study also identified a *confidence trap*: AI agents introduced breaking changes even when they reported the highest confidence [8]. Both tools focus on structural changes and do not analyze behavior inside method bodies.

Kong et al. studied breaking changes in the NPM ecosystem that developers themselves acknowledged in commit messages: from 5,242 breaking-change-related commits they built a dataset of 1,519 breaking changes from 131 projects and analyzed why and how they are

introduced (for example, reducing redundancy, improving identifier names and improving API design). Of the breaking changes detected in their sample, 95.4% were documented by developers, while about 19% of breaking changes could not be detected by regression testing [21]. The study extends the change patterns beyond Python to JavaScript, including changes to exported members, which motivates the ExportAnalyzer component, and shows that regression tests alone are not enough to reveal all breaking changes.

I.6.3 Code–Documentation Inconsistency Detection

DocPrism uses a standard LLM with the Local Categorization, External Filtering (LCEF) methodology: the prompt asks local yes/no questions per category inside a JSON schema, and under-promises are removed after generation. In its ablation, LCEF reduced the flag rate from 98% to 14% and increased F1 from 0.22 to 0.77; across Python, TypeScript, C++ and Java it kept a 17% flag rate with 0.63 precision [6].

Cascade generates unit tests from documentation and executes them on the code; to reduce false positives, it also generates an implementation from the documentation and reports an inconsistency only when the original code fails a test that the generated code passes. On a dataset of 71 real, developer-confirmed inconsistencies and 814 consistent methods, it reached 0.88 precision and 0.97 specificity with 0.21 recall on the balanced split, and 0.39 precision on the realistic 10/90 split, higher than all compared tools, with only 31 false positives among 814 consistent methods [4]. METAMON asks an LLM whether EvoSuite regression assertions agree with the documentation and uses metamorphic (negated) queries and self-consistency, reaching 0.72 precision and 0.48 recall on 9,482 pairs [9]. Kang et al. showed that about one fifth of GPT-4-generated comments contained inaccurate statements and that nine existing detectors had no significant relationship with accuracy, while their document-testing approach did (ROC-AUC 0.67) [10].

C4RLLaMA fine-tunes CodeLLaMA-7B (with LoRA and a dedicated loss) to detect a comment inconsistency and rectify the comment in one step. On the Java dataset of Panthaplackel et al. (40,688 samples) it reports, in the post-hoc setting, a precision of 94.0%, recall of 84.6% and F1 of 89.0% [5]. However, when DocPrism applied it to real code-documentation pairs, it flagged every function (100% flag rate) and its precision fell to 0.08 [6]. It is therefore used here as a cautionary reference against evaluating only on synthetic data.

I.6.4 Logical Code Explanation and Its Evaluation

Code explanation (code summarization or explanatory documentation) describes what code does and why. CodeExp studied what makes an explanation useful: a human study defined four quality criteria (correctness, informativeness, coherence and fluency), and the authors collected 2.285M code–docstring pairs, refined them to 158,024 and human-annotated 13,186; training on the refined data outperformed training on the raw data [24]. Most summarization work is evaluated with reference-based metrics such as BLEU-4 and METEOR. Makharev and Ivanov extended summarization beyond single functions to class and repository level using modified ClassEval (400 samples) and modified CodeSearchNet (806 samples); fine-tuned CodeT5+ performed best, few-shot prompting and retrieval of repository snippets (RAG) substantially improved LLM results, and repository-level summarization benefited from structured context [25].

Reference-based metrics measure similarity to a human text, not factual accuracy. Kang et al. showed that about one fifth of GPT-4-generated method comments contained demonstrably inaccurate statements, and that BLEU, SentenceBERT and existing consistency detectors had no significant relationship with this accuracy [10]. ETF addresses hallucination directly: it extracts entities from the code with static analysis, then uses an LLM to locate and verify these entities in the generated summary; on CodeSumEval (411 summaries from 7 LLMs with 9,933 entity-level annotations) it reaches a 73% F1 score in detecting hallucinations [23]. For class-level code, ClassEval provides a manually crafted benchmark of 100 class-level tasks (412 methods, 33.1 tests per class on average) and showed that class-level generation is harder than method-level generation [22]; it is used here as a reference for evaluating explanations at class level.

Work	Approach	Data	Evaluation / result	Use in this project
CodeExp (Findings of EMNLP 2022) [24]	Human study of explanation quality; corpus refinement; multi-stage fine-tuning	2.285M raw, 158,024 refined, 13,186 human-annotated pairs	Correctness, informativeness, coherence, fluency; BLEU, METEOR	Defines explanation-quality criteria
ETF (ACL 2025) [23]	Static entity extraction + LLM entity verification	CodeSumEval: 411 summaries, 9,933 entity annotations	73% F1 for hallucination detection	Basis of entity/claim-level verification
ClassEval (ICSE 2024) [22]	Manually crafted class-level benchmark	100 classes, 412 methods, 33.1 tests/class	Pass@k; class level harder than method level	Reference for class-level evaluation
Makharev & Ivanov (LLM4Code 2025) [25]	Function/class/ repository summarization with few-shot and RAG	400 modified ClassEval + 806 modified CodeSearchNet	BLEU-4, METEOR, ROUGE-L, BERTScore, BLEURT, SIDE	Extends evaluation from function to file/project level
Kang et al. (2024) [10]	Document testing of LLM-generated comments	540 generated Java comments	~1/5 of GPT-4 comments inaccurate; ROC-AUC 0.67	Shows similarity metrics do not measure accuracy

Table 6: State of the art in code explanation and its evaluation

I.6.5 Comparison and Identified Gap

Work	Track	Approach	Main result	Limitation
AexPy (ISSRE 2022) [7]	Code vs Code	Hybrid API model, 42 patterns, 4 severity levels	More known breaking changes found than PyCompat and pidiff	Needs an installable package; structural only
Safer Builders (MSR 2026) [8]	Code vs Code	AST of reconstructed hunks, 17 patterns	93.6–95.7% TP, κ = 0.79; confidence trap	Structural only
DocPrism (ISSTA 2026) [6]	Code vs Doc	LLM + LCEF	Flag rate 98% → 14%, F1 0.22 → 0.77	No deterministic pre-check; 48% of FPs from incorrect reasoning
Cascade (FSE 2026) [4]	Code vs Doc	Doc-derived tests + generated reference code	Precision 0.88, specificity 0.97, recall 0.21	Needs compilable, executable projects; low recall
METAMON (LLM4Code 2025)	Code vs Doc	EvoSuite + metamorphic LLM	Precision 0.72, recall 0.48	Java/EvoSuite only; mutant-based data

Work	Track	Approach	Main result	Limitation
[9]		queries		
Kang et al. (2024) [10]	Generated docs	Document testing	ROC-AUC 0.67	Needs test execution environment
C4RLLaMA (ICSE 2025) [5]	Code vs Doc	Fine-tuned CodeLLaMA-7B: detect + rectify	Post hoc P 94.0%, R 84.6%, F1 89.0% (synthetic)	Precision 0.08 and 100% flag rate on real data
NPM breaking changes (TOSEM 2025) [21]	Code vs Code	Empirical study of 1,519 acknowledged breaking changes	95.4% documented; ~19% not detectable by regression tests	Empirical study, no detection tool

Table 7: State of the art in conflict and inconsistency detection

Identified gap. Among the reviewed works, breaking-change detectors are structural and do not analyze behavior, the most precise documentation checkers require executing the analyzed project, and LLM-only checkers lack a deterministic pre-filter. For explanations, the common reference-based metrics do not measure factual accuracy, and entity-level verification (ETF) does not check behavioral claims such as return values or exceptions. No reviewed system combines (i) deterministic AST analysis before the LLM, (ii) an evidence check after the LLM, (iii) generated tests that are accepted only after passing in a sandbox on the current code, with commit-based coverage history and failure classification, and (iv) claim-level verification of generated explanations against the AST, within one platform covering Code vs Code, Code vs Documentation, explanation and testing. This project addresses that gap.

I.7 Proposed Solution

I.7.1 Solution Idea

The solution follows three principles:

1. **Deterministic first.** Everything that can be decided from the syntax tree (renames, parameter changes, element type changes, exports, severity) is decided by AST analysis, which is exact and free.
2. **LLM only where semantics are needed.** The LLM is called for elements whose bodies changed (behavioral candidates), for per-element documentation checks, for generating explanations, and for test generation.
3. **Evidence before trust.** LLM output is accepted only with verifiable evidence: quoted code that exists in the analyzed body, a confidence above a threshold, explanation claims that match the AST, or tests that compile and run in the sandbox.

The improvements are evaluated against a frozen copy of the phase-one system using a pinned LLM, so that every reported gain is measurable and reproducible.

I.7.2 Block Diagram

Figure 1 shows the main components. The analysis layer converts all inputs into a unified representation that now includes parameters (kind, type, default), decorators, a normalized body hash and docstrings. The intelligence layer contains four engines that share this representation (Code vs Code, Code vs Documentation, test generation, and the explanation engine with the HighLevelAgent and LowLevelAgent) and a single LLM service with a

pinned model and per-user quota. The verification and execution layer validates and repairs generated tests and executes them in an isolated sandbox, checks the evidence and confidence of LLM findings, and verifies the claims of generated explanations against the AST through the VerifierAgent. Verified explanations are also passed to the Code vs Documentation engine as generated documentation.

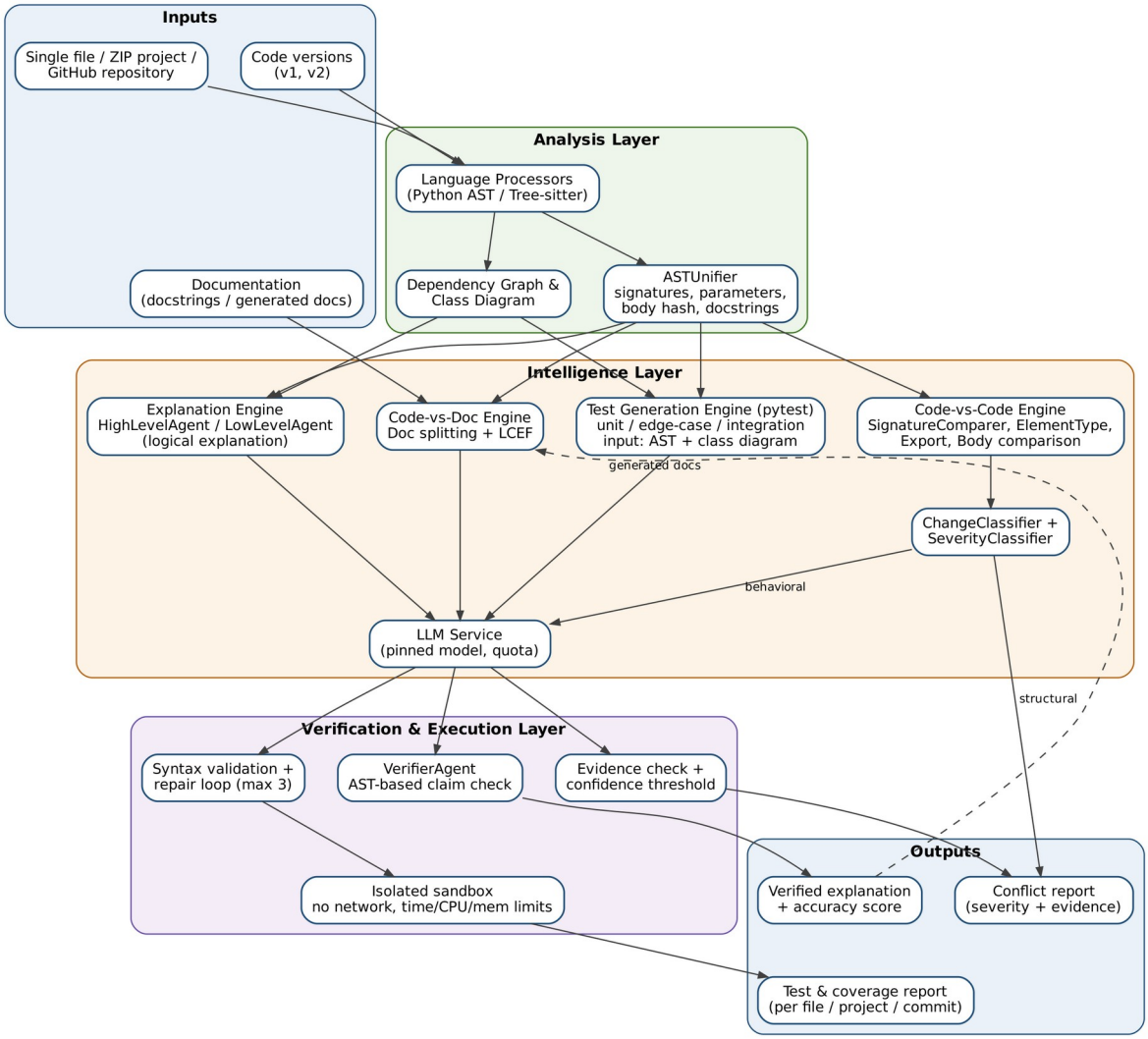


Figure 1: Block diagram of the proposed system

I.7.3 Flowchart: Test Generation and Execution

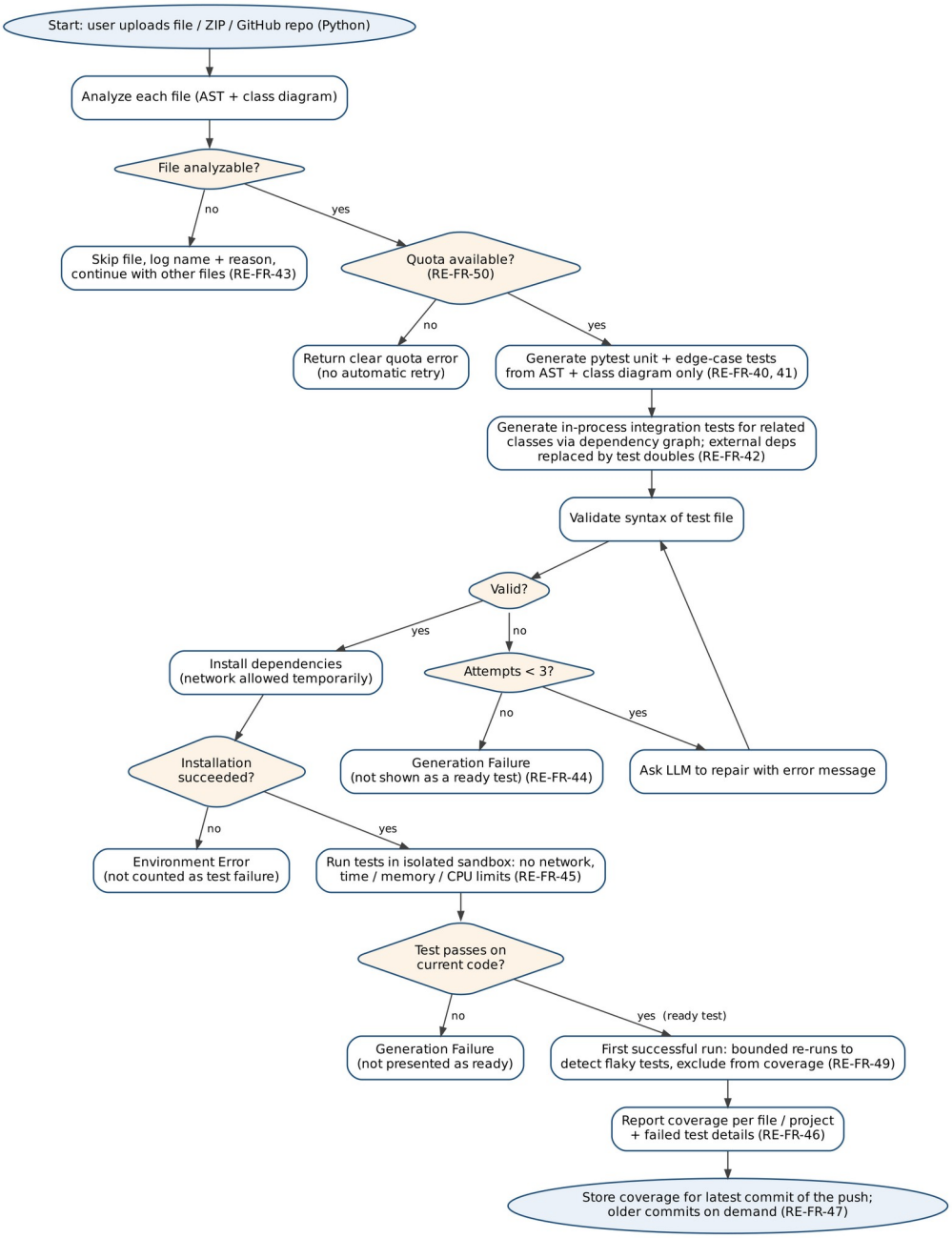


Figure 2: Test generation and execution flow

I.7.4 Flowchart: Code vs Code Analysis

The ChangeClassifier combines the structural result with the body comparison using the decision matrix in Table 8. Body comparison is always executed, even when the signature changed, because behavioral changes can hide behind structural ones.

Signature	Body	Decision	LLM call?
Changed	Unchanged	structural_only	No
Changed	Changed	structural + behavioral	Yes, with structural context
Unchanged	Changed	behavioral_candidate	Yes
Unchanged	Unchanged	no_change	No (stop)

Table 8: Decision matrix of the ChangeClassifier

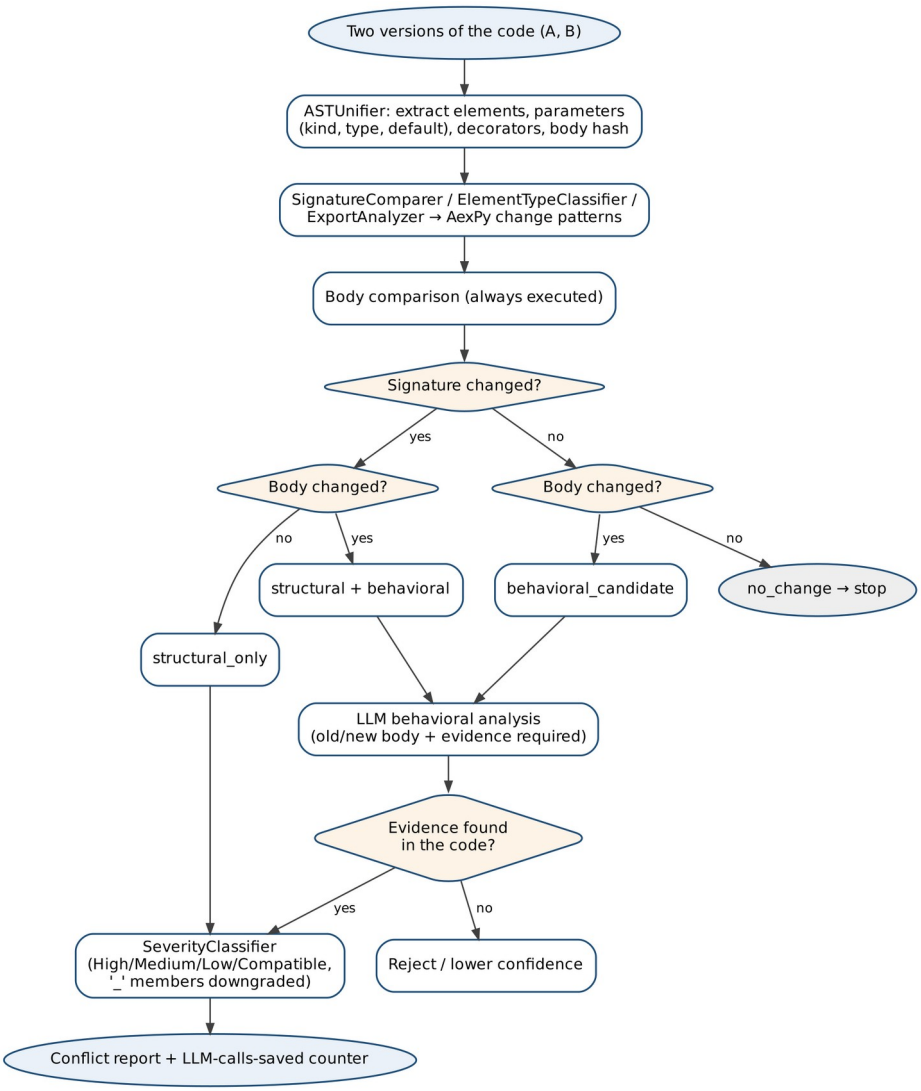


Figure 3: Code vs Code analysis flow

Severity is assigned from the AexPy level of each pattern: High maps to *critical*, Medium and Low map to *warning*, and Compatible maps to *informational*; members whose names start with an underscore are downgraded to informational. Behavioral changes take their severity from the LLM, but are raised to critical only when the confidence threshold and the evidence check are both satisfied.

I.7.5 Flowchart: Code vs Documentation Analysis

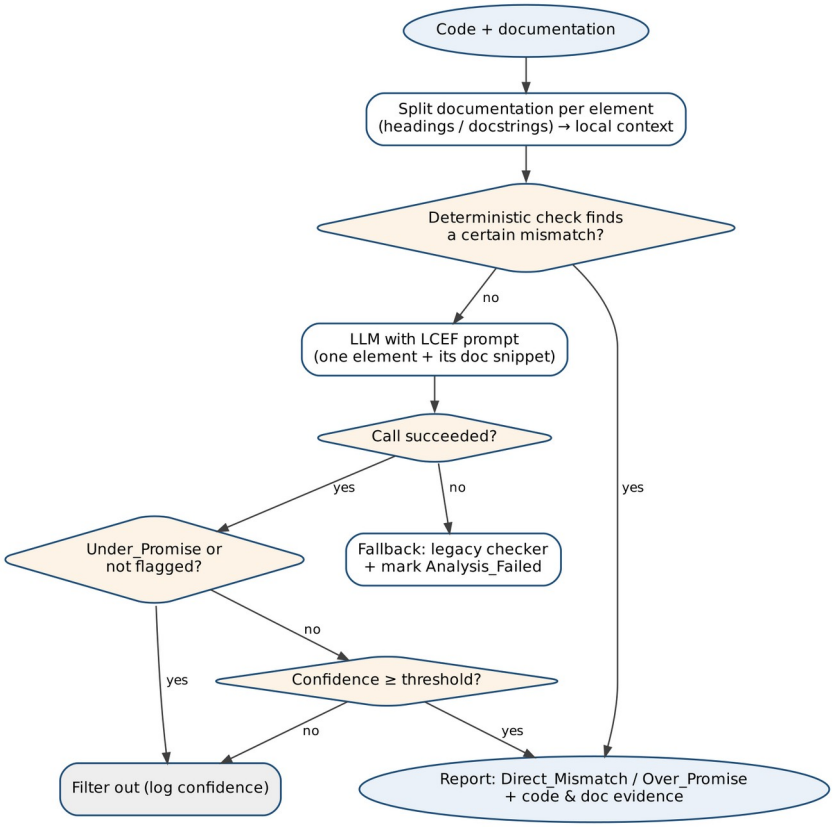


Figure 4: Code vs Documentation analysis flow

Undocumented elements are reported as documentation coverage statistics rather than inconsistencies, because missing documentation is an incompleteness issue, not an incorrectness issue.

I.7.6 Flowchart: Logical Code Explanation

Figure 5 shows how explanations are generated and verified. Context is built in dependency order, and when it exceeds the model's token limit, dependencies are represented by their signatures only. The HighLevelAgent produces a purpose- and architecture-level explanation, while the LowLevelAgent describes methods, parameters, return values, exceptions and logic flow. Following the entity-tracing idea of ETF [23], the explanation is then decomposed into entities and claims. Entities that do not exist in the AST are flagged as hallucinations; structural claims (signatures, return types, raised exceptions, calls) are checked deterministically against the AST; and behavioral claims are verified by the LLM only with quoted code evidence. The share of verified claims gives the explanation's accuracy score. When incorrect claims are found, the VerifierAgent corrects only the flagged statements, for at most two attempts, instead of rewriting the whole document.

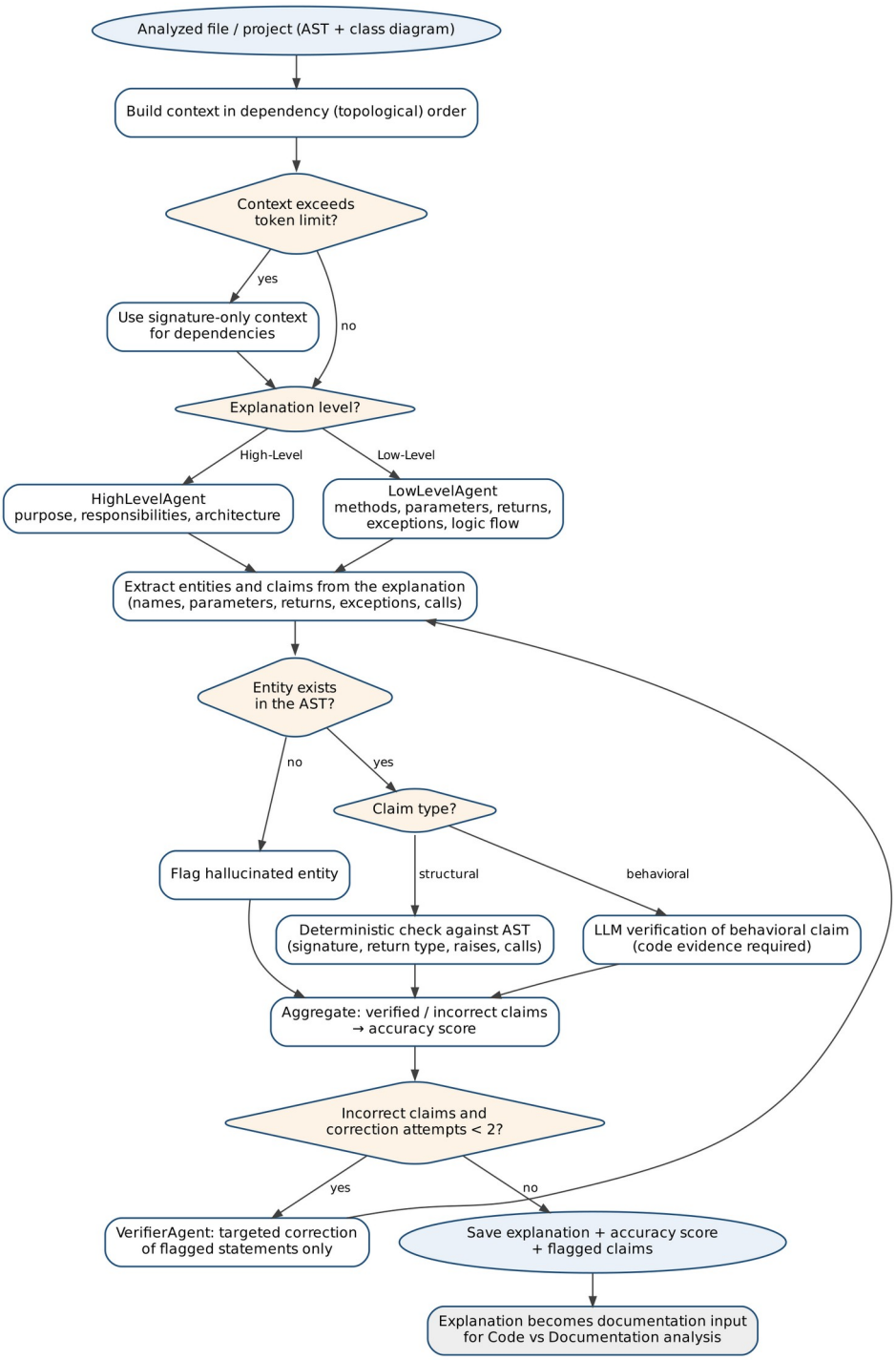


Figure 5: Logical code explanation and verification flow

Chapter 1: Theoretical Study

1.1 Introduction

This chapter presents the theoretical concepts on which the project is built and which are needed during implementation: software testing and automated test generation, test adequacy, safe execution, abstract syntax trees, API evolution and breaking changes, code–documentation inconsistency, logical code explanation and its evaluation, the use of LLMs in software engineering, and the methodology used to evaluate the system.

1.2 Software Testing Fundamentals

Software testing executes a program with selected inputs and compares the observed behavior with the expected behavior. Tests are usually organized by level:

- **Unit tests** exercise a single function, method or class in isolation, often replacing collaborators with test doubles (mocks or stubs).
- **Integration tests** exercise several units together to check that their interfaces and interactions work as intended. In this project, related units are identified through the dependency graph and class-diagram relations; integration is tested in-process, while external dependencies are replaced by test doubles.
- **System and acceptance tests** check the complete application against its requirements.

A test consists of a *prefix* (setup and inputs) and an *oracle* (assertions on the result). Deciding the expected result automatically is known as the **oracle problem**; it is the main difficulty of automated test generation, because an assertion derived from the current code captures what the code does, not necessarily what it should do.

Two classical black-box techniques guide input selection. **Equivalence partitioning** divides the input domain into classes expected to behave the same, so one representative per class is enough. **Boundary value analysis** adds inputs at and around the edges of these classes (for example null, empty, zero, negative and maximum values), where defects are most frequent. Edge-case tests in RE-FR-41 apply these techniques together with the exceptions identified in the code.

1.3 Automated Test Generation

Automated test generation techniques fall into three families:

- **Random and feedback-directed generation** (for example Randoop) builds call sequences randomly and uses execution feedback to discard invalid or redundant sequences [11].
- **Search-based generation** (for example EvoSuite and Pynguin) treats test generation as an optimization problem, typically evolving test suites with genetic algorithms to maximize a coverage criterion [12], [13].
- **LLM-based generation** prompts a language model with information about the code under test (in this project, the AST and class-diagram data rather than the whole project source), and asks it to write tests in the target framework (pytest, JUnit). Feedback loops improve the result: compilation or runtime errors are returned to the model for

repair (TestPilot [1]), coverage gaps guide further generation (CoverUp [2]), and only tests that build, pass and add value are kept (TestGen-LLM [3]).

The project does not use these tools; it implements its own LLM-based pipeline and adopts safeguards motivated by the reviewed works: a structured focal context around the unit under test (ChatUniTest [33]), a syntax validation step with a bounded repair loop of at most three attempts that returns the error message to the model (TestPilot [1]), and acceptance based on execution results before tests are reported (CoverUp [2], TestGen-LLM [3]). Mutation testing, in which surviving mutants can also be fed back to the model (MuTAP [34]), is used to assess the quality of accepted tests.

1.4 Test Adequacy: Coverage and Mutation Testing

Code coverage measures how much of the program a test suite executes. *Line (statement) coverage* is the fraction of executable lines that run at least once; *branch coverage* is the fraction of decision outcomes (true/false of each condition) that are taken. In Python, coverage is commonly collected with coverage.py through pytest-cov.

$$\textit{Line coverage} = \textit{executed lines} / \textit{executable lines} \qquad \textit{Branch coverage} = \textit{taken branches} / \textit{total branches}$$

Coverage shows which code is executed but not whether the assertions are strong. **Mutation testing** addresses this by creating *mutants*, small artificial faults such as replacing an operator or a constant, and checking whether the test suite fails on them ("kills" them) [15]. The mutation score estimates the fault-detection ability of the suite:

$$\textit{Mutation score} = \textit{killed mutants} / (\textit{total mutants} - \textit{equivalent mutants})$$

Mutation testing is computationally expensive, which is why RE-FR-51 is planned as a stretch goal.

1.5 Safe Execution of Generated Code

Generated tests import and execute the user's project code, which is untrusted from the platform's point of view. **Sandboxing** isolates this execution from the host. Container-based isolation (for example Docker) gives each run its own file system and process space, and operating-system controls enforce limits on execution time, memory and CPU. Disabling network access prevents data exfiltration and makes results independent of external services. Because tests cannot download packages during execution, dependencies declared by the project are installed in a separate, controlled preparation step in which network access is allowed temporarily, before the isolated run without network (RE-FR-45). Failures in this step are environment errors, which must be distinguished from failures of the generated tests themselves.

1.6 Regression Test Selection and Flaky Tests

Regression test selection (RTS) runs only the tests that may be affected by a change instead of the entire suite. Static RTS uses a dependency graph: if a changed module is reachable from a test's dependencies, the test is selected [16]. Limiting the search depth in the graph keeps the selection cheap. The platform already builds a dependency graph during analysis, which is reused for RE-FR-48.

A **flaky test** passes and fails on the same code without any change, typically because of asynchronous waits, concurrency, test-order dependency, time, randomness or external resources [17]. Flaky tests distort coverage and erode trust. A common detection strategy is to re-run tests several times on unchanged code and mark tests with inconsistent outcomes (RE-FR-49); in this project the check runs once, at a test's first successful run, with a fixed number of re-runs.

1.7 Abstract Syntax Trees and Static Analysis

An **Abstract Syntax Tree (AST)** is a tree representation of the syntactic structure of source code in which each node is a construct such as a module, class, function, argument or expression. Unlike plain text, an AST ignores formatting and comments, so it supports precise structural analysis. Python's standard `ast` module parses Python code into such a tree, and functions such as `ast.get_docstring` and `ast.get_source_segment` retrieve a function's docstring and source. **Tree-sitter** is an incremental parser generator that provides concrete syntax trees for many languages and is used by the platform for Java and other languages.

Two AST techniques are central to this project:

- **Normalized body comparison.** Two versions of a function body are compared by hashing a normalized dump of their ASTs after removing the docstring. Changes in whitespace, comments or documentation do not alter the hash, while any change in logic does.
- **Hunk reconstruction.** A Git diff is made of *hunks*, contiguous blocks of added and removed lines, which cannot be parsed directly. Safer Builders reconstructs syntactically valid before and after versions of each hunk so they can be parsed into ASTs without loading the whole project [8].

Dependency graphs and topological ordering. A dependency graph is a directed graph whose nodes are code elements (modules, classes or functions) and whose edges represent relations such as imports, inheritance and calls. Three operations on this graph are used in the project. First, a *topological order* lists every element after the elements it depends on (for example with Kahn's algorithm); it determines the order in which explanation context is built (Figure 5). Topological ordering requires an acyclic graph, while circular imports are common in Python, so cycles must be handled, for example by condensing each strongly connected component into a single node. Second, groups of connected elements identify candidates for integration tests (RE-FR-42). Third, a reverse traversal from a changed element, limited to a fixed depth, finds the elements and tests that may be affected by a change (RE-FR-48, Section 1.6).

1.8 API Evolution and Breaking Changes

A **breaking change** is a modification that violates backward compatibility, so that client code which worked with the previous version fails or behaves incorrectly with the new one. Semantic Versioning signals such changes by incrementing the major version number [18]. Breaking changes are either *structural* (removing or renaming an API, changing parameters or return types, changing class hierarchies) or *behavioral* (the same interface produces different results, raises different exceptions, or has different side effects).

Python makes breaking-change detection difficult because of dynamic typing and flexible argument passing. A parameter can be positional-only, positional-or-keyword, keyword-only, variadic positional (*args) or variadic keyword (**kwargs), and it may have a default value. Whether a change breaks a caller depends on this kind: adding a parameter with a default is compatible, while adding a required parameter, removing a parameter, reordering positional parameters, or removing a default is breaking. AexPy formalizes 42 such change patterns and grades them into four levels (Table 9) [7].

AexPy level	Meaning	Example	System output
High	Directly breaks usage	Remove an API or a base class; reorder parameters	critical
Medium	Limited impact	Parameter type no longer accepts a previously valid type	warning
Low	Fake-private members or external references	Add an optional parameter to a function whose name starts with _	warning
Compatible	Safe change	Return type becomes a subclass of the old type	informational

Table 9: AexPy severity levels and their mapping in the system

Python has no enforced access control: a leading underscore only marks a member as private by convention, and the same object may be reachable through several names (aliases). Changes to underscore members are therefore downgraded, and alias tracking prevents reporting a removal when the old name is still available.

1.9 Code–Documentation Inconsistency

Method-level documentation acts as a specification of what a method does. A **code–documentation inconsistency** exists when the documentation and the implementation disagree. Following DocPrism [6], three categories are distinguished:

- **Direct mismatch:** the documentation states something specific and the code does something different at the same point (for example, the documentation says a method raises KeyError while the code returns None).
- **Over-promise:** the documentation claims a behavior that the code does not implement at all (for example, "validates the email format" with no validation).
- **Under-promise:** the code does something that the documentation does not mention. Because documentation is naturally more concise than code, under-promises are *incompleteness* issues, whereas direct mismatches and over-promises are *incorrectness* issues.

Detection can be **just-in-time**, using a code change (diff) as additional input, or **post hoc**, analyzing code and documentation as they exist. The project uses the post-hoc setting. Evaluation data must be chosen carefully: synthetic datasets that pair old documentation with new code contain noisy labels and do not reflect real inconsistencies, which explains the large performance drops reported when tools trained on them are applied to real code [6].

1.10 Logical Code Explanation and Summarization

Code explanation (also called code summarization or explanatory documentation) is natural-language text that describes what a piece of code does, how it does it and why. Explanations can be produced at several levels of granularity: function or method level, class level, and file, project or repository level [25]. In the platform, the **High-Level** explanation targets managers and reviewers (purpose, responsibilities, architecture), while the **Low-Level** explanation targets developers (methods, parameters, return values, exceptions and logic flow).

Quality criteria. Following CodeExp [24], a good explanation is *correct* (it describes the code truthfully), *informative* (it gives useful information beyond restating names), *coherent* (its parts are logically connected) and *fluent* (it is well written). Correctness is the most critical criterion, because a fluent but wrong explanation misleads the reader.

Reference-based metrics. Summaries are often scored by comparing them with a human-written reference: BLEU-4 counts overlapping n-grams, METEOR also matches stems and synonyms, ROUGE-L uses the longest common subsequence, and BERTScore and BLEURT compare contextual embeddings [25]. These metrics need a reference text and measure similarity, not truth: changing a single word (for example *raises* to *returns None*) barely changes the score but reverses the meaning [10].

Hallucination and entity tracing. A *hallucination* in an explanation is a statement that is not supported by the code, such as a non-existent method or a wrong description of behavior. Entity tracing detects such errors by extracting the entities of the code (classes, methods, parameters, variables) with static analysis and checking how each entity is described in the explanation [23]. This project extends the idea from entities to *claims*: each claim about a signature, return value, exception or call is checked deterministically against the AST, and only behavioral claims are sent to the LLM with required code evidence.

Claim-level accuracy. For evaluation, each explanation is split into atomic claims, and annotators label each claim as correct or incorrect against the code. Two measures are derived:

$$\textit{Factual precision} = \textit{correct claims} / \textit{all claims} \qquad \textit{Inaccurate-explanation rate} = \textit{explanations with} \geq 1 \textit{ incorrect claim} / \textit{all explanations}$$

1.11 Large Language Models in Software Engineering

1.11.1 Prompt Engineering

LLMs are controlled through prompts. *Zero-shot* prompts describe the task only, while *few-shot* prompts add solved examples. *Chain-of-Thought* prompting asks the model to reason step by step before answering. *Structured output* requires a fixed JSON schema, which makes responses machine-readable. DocPrism's **Local Categorization** embeds each decision as a local yes/no question inside the JSON schema, so the model completes the format instead of remembering long definitions, and **External Filtering** removes unwanted categories in code after generation rather than asking the model to omit them [6].

1.11.2 Reliability of LLM Output

- **Hallucination:** models may produce fluent but false statements, invent APIs, or assert behavior that does not exist.
- **Self-consistency and metamorphic queries:** asking the same question several times, or asking a logically inverted question, and checking that answers agree increases reliability; METAMON uses negated assertions as metamorphic relations [9].
- **The confidence trap:** a model's declared confidence does not guarantee correctness [8]. The project therefore requires evidence (quoted code that exists in the analyzed body) and applies confidence thresholds as an additional filter, not as proof.
- **Execution as verification:** executing tests derived from documentation provides objective evidence (Cascade [4], Kang et al. [10]); the same principle underlies the validation and sandboxed execution of generated tests.

1.11.3 Determinism, Cost and Quotas

LLM output depends on the model, its version and sampling parameters. For reproducible evaluation, the temperature is set to 0, a fixed seed is used where supported, caching is disabled, and a specific model is pinned instead of a router that switches models. LLM usage is billed and limited in *tokens*, the units into which text is split; input and output tokens are counted separately, which is the basis of the per-user quota in RE-FR-50.

1.11.4 LLM Agents and Multi-Agent Design

An **LLM agent** is an LLM call wrapped with a defined role, role-specific instructions, a structured input and an expected output format, and possibly access to tools such as a parser or a test runner. A **multi-agent design** divides a complex task among several specialized agents instead of asking one prompt to do everything, which keeps each prompt focused and makes intermediate results inspectable.

The platform uses this design for explanations: the HighLevelAgent and LowLevelAgent generate text at two levels of abstraction, and the VerifierAgent checks it. This follows the *generator–verifier* pattern, in which one component produces output and another accepts, rejects or corrects it. A verifier that is itself an LLM may share the generator's blind spots and is subject to the confidence trap [8]. For this reason, the VerifierAgent relies on deterministic AST checks for structural claims and requires quoted code evidence for behavioral claims [23]. The number of correction rounds is also bounded (at most two attempts), which keeps cost predictable and prevents endless loops.

1.12 Evaluation Methodology

1.12.1 Classification Metrics

Detection is evaluated as a classification task using the counts of true positives (TP), false positives (FP), true negatives (TN) and false negatives (FN):

Metric	Definition	Question answered
Precision	TP / (TP + FP)	Of all reported issues, how many are real?
Recall	TP / (TP + FN)	Of all real issues, how many were found?
Specificity	TN / (TN + FP)	Of all correct elements, how many were

Metric	Definition	Question answered
		left alone?
F1 score	2 · Precision · Recall / (Precision + Recall)	Balance between precision and recall
Flag rate	Flagged elements / all elements	How noisy is the tool for developers?
Under-promise rate	Under-promises / all reported inconsistencies	Is incompleteness leaking into the output?
PFP	Detected inconsistent items whose fixed version is classified consistent / detected items	Does the tool detect the inconsistency itself or a superficial pattern?

Table 10: Evaluation metrics

Class imbalance. In real projects most elements are consistent, so precision drops when the share of negatives grows. Cascade therefore reports results on a balanced 50/50 split and on a realistic 10/90 split, taking the median over many random samples [4]; the same protocol is used in this project.

1.12.2 Ground Truth and Inter-Annotator Agreement

Metrics are only as reliable as the labels they are computed against. Each sample is labeled independently by two annotators following a written guideline, and agreement is measured with **Cohen's kappa** [19], which corrects the observed agreement for agreement expected by chance:

$$\kappa = (p_o - p_e) / (1 - p_e)$$

where p_o is the observed agreement and p_e is the chance agreement computed from each annotator's label frequencies. Following Landis and Koch [20], κ between 0.61 and 0.80 indicates substantial agreement and above 0.80 almost perfect agreement; the project targets $\kappa \geq 0.60$ before disagreements are resolved by discussion. The labeling questions are defined by behavior observable to a caller rather than by the system's own rules, so that the evaluation does not simply mirror the implementation.

1.12.3 Reproducible Baseline Comparison

To measure improvement, the phase-one detection code and language processors are frozen in a separate package, with a SHA-256 fingerprint of every file stored in a manifest so that any modification is detected before an evaluation run. The frozen baseline and the improved system are run on the same samples with the same pinned model, and all raw responses, model versions and settings are recorded.

1.13 Summary

This chapter introduced the concepts used throughout the project: test levels, the oracle problem and test design techniques; random, search-based and LLM-based test generation; coverage and mutation testing; sandboxed execution, regression test selection and flaky tests; ASTs, body hashing, hunk reconstruction, dependency graphs and topological ordering; breaking changes and the AexPy severity model; the taxonomy of code–documentation inconsistencies; explanation levels, quality criteria, reference-based metrics, hallucination and claim-level accuracy; prompt engineering, the reliability limits of LLMs, and LLM agents with the generator–verifier pattern; and the metrics and agreement measures used to evaluate the system.

References

[1] Schäfer, M., Nadi, S., Eghbali, A., & Tip, F. (2024). An empirical evaluation of using large language models for automated unit test generation. IEEE Transactions on Software Engineering, 50(1), 85–105.

[2] Pizzorno, J. A., & Berger, E. D. (2025). CoverUp: Effective high coverage test generation for Python. Proceedings of the ACM on Software Engineering (FSE 2025).

[3] Alshahwan, N., et al. (2024). Automated unit test improvement using large language models at Meta. In FSE 2024 Companion (Industry Track).

[4] Kiecker, T., Sparka, J. A., Reuter, M., Ziegler, A., & Grunske, L. (2026). Cascade: Detecting inconsistencies between code and documentation with automatic test generation. Proc. ACM Softw. Eng., 3(FSE), Article FSE168.

[5] Rong, G., Yu, Y., Liu, S., Tan, X., Zhang, T., Shen, H., & Hu, J. (2025). Code comment inconsistency detection and rectification using a large language model. In 47th IEEE/ACM International Conference on Software Engineering (ICSE 2025).

[6] Xu, X., Wahab, Z., Holmes, R., & Lemieux, C. (2026). DocPrism: Multi-lingual detection of incorrectness inconsistencies between code and documentation. Proc. ACM Softw. Eng., 3(ISSTA), Article ISSTA157.

[7] Du, X., & Ma, J. (2022). AexPy: Detecting API breaking changes in Python packages. In IEEE 33rd International Symposium on Software Reliability Engineering (ISSRE), 470–481.

[8] Ferdous, K. M., Banik, D., Chowdhury, K., & Shamim, S. I. (2026). Safer builders, risky maintainers: A comparative study of breaking changes in human vs agentic PRs. In MSR '26.

[9] Lee, H., An, G., & Yoo, S. (2025). Metamon: Finding inconsistencies between program documentation and behavior using metamorphic LLM queries. In LLM4Code @ ICSE 2025, 120–127.

[10] Kang, S., Milliken, L., & Yoo, S. (2024). Identifying inaccurate descriptions in LLM-generated code comments via test execution. arXiv:2406.14836.

[11] Pacheco, C., & Ernst, M. D. (2007). Randoop: Feedback-directed random testing for Java. In OOPSLA Companion, 815–816.

[12] Fraser, G., & Arcuri, A. (2011). EvoSuite: Automatic test suite generation for object-oriented software. In ESEC/FSE '11, 416–419.

[13] Lukasczyk, S., Kroiß, F., & Fraser, G. (2020). Automated unit test generation for Python. In Search-Based Software Engineering (SSBSE 2020). arXiv:2007.14049.

[14] Lemieux, C., Inala, J. P., Lahiri, S. K., & Sen, S. (2023). CodaMosa: Escaping coverage plateaus in test generation with pre-trained large language models. In ICSE 2023.

[15] Jia, Y., & Harman, M. (2011). An analysis and survey of the development of mutation testing. IEEE Transactions on Software Engineering, 37(5), 649–678.

[16] Gligoric, M., Eloussi, L., & Marinov, D. (2015). Practical regression test selection with dynamic file dependencies. In ISSTA 2015, 211–222.

[17] Luo, Q., Hariri, F., Eloussi, L., & Marinov, D. (2014). An empirical analysis of flaky tests. In FSE 2014, 643–653.

[18] Preston-Werner, T. Semantic Versioning 2.0.0. https://semver.org

[19] Cohen, J. (1960). A coefficient of agreement for nominal scales. Educational and Psychological Measurement, 20(1), 37–46.

[20] Landis, J. R., & Koch, G. G. (1977). The measurement of observer agreement for categorical data. Biometrics, 33(1), 159–174.

[21] Kong, D., Liu, J., Bao, L., & Lo, D. (2025). Towards better comprehension of breaking changes in the NPM ecosystem. ACM Transactions on Software Engineering and Methodology (TOSEM).

[22] Du, X., Liu, M., Wang, K., Wang, H., Liu, J., Chen, Y., Feng, J., Sha, C., Peng, X., & Lou, Y. (2024). Evaluating large language models in class-level code generation. In 46th IEEE/ACM International Conference on Software Engineering (ICSE 2024).

[23] Maharaj, K., Munigala, V., Tamilselvam, S. G., Kumar, P., Sen, S., Kodeswaran, P., Mishra, A., & Bhattacharyya, P. (2025). ETF: An entity tracing framework for hallucination detection in code summaries. In Proceedings of ACL 2025, 30639–30652.

[24] Cui, H., Wang, C., Huang, J., Inala, J. P., Mytkowicz, T., Wang, B., Gao, J., & Duan, N. (2022). CodeExp: Explanatory code document generation. In Findings of EMNLP 2022.

[25] Makharev, V., & Ivanov, V. (2025). Code summarization beyond function level. In IEEE/ACM LLM4Code Workshop, co-located with ICSE 2025.

[26] Qodo. Qodo Gen and Qodo Cover (formerly CodiumAI / Cover-Agent). https://www.qodo.ai (accessed October 2026).

[27] Diffblue. Diffblue Cover and Diffblue Testing Agent documentation. https://docs.diffblue.com (accessed October 2026).

[28] Codecov. Code coverage reporting. https://about.codecov.io (accessed October 2026).

[29] CloudBees. CloudBees Smart Tests. https://www.cloudbees.com (accessed October 2026).

[30] GitHub. GitHub Copilot. https://github.com/features/copilot (accessed October 2026).

[31] Keploy. Keploy: API and integration testing agent. https://github.com/keploy/keploy (accessed October 2026).

[32] mabl. Auto-healing tests. https://www.mabl.com/auto-healing-tests (accessed October 2026).

[33] Chen, Y., Hu, Z., Zhi, C., Han, J., Deng, S., & Yin, J. (2024). ChatUniTest: A framework for LLM-based test generation. In FSE 2024 Companion (Demonstrations). arXiv:2305.04764.

[34] Dakhel, A. M., Nikanjam, A., Majdinasab, V., Khomh, F., & Desmarais, M. C. (2024). Effective test generation using pre-trained large language models and mutation testing. Information and Software Technology. arXiv:2308.16557.

[35] Huang, D., Zhang, J. M., Harman, M., Zhang, Q., Du, M., & Ng, S.-K. (2025). Benchmarking LLMs for unit test generation from real-world functions. arXiv:2508.00408.

[36] Wang, Y., Xia, C., Zhao, W., Du, J., Miao, C., Deng, Z., Yu, P. S., & Xing, C. (2025). MultiFileTest (formerly ProjectTest): A multi-file-level LLM unit test generation benchmark and impact of error fixing mechanisms. arXiv:2502.06556.